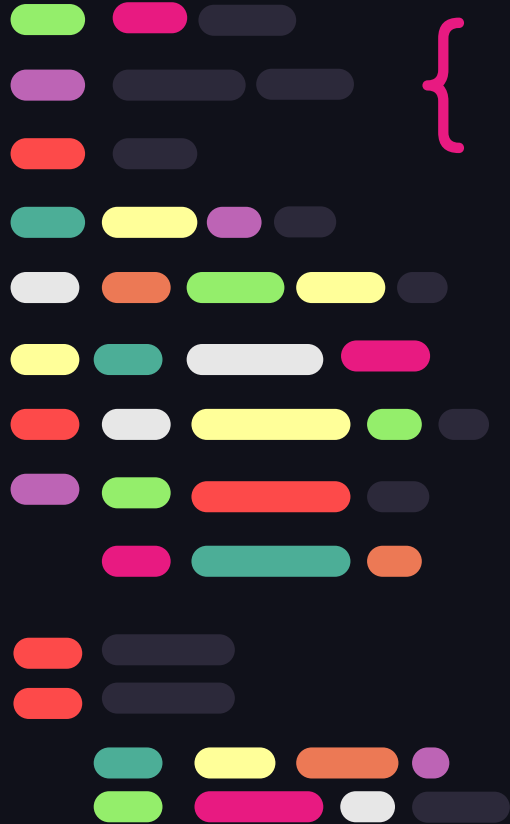




# The Hitchhiker's Guide to **Cursor** (You Don't Need To Bring a Towel for This)

< Advanced Session | June 2026 >

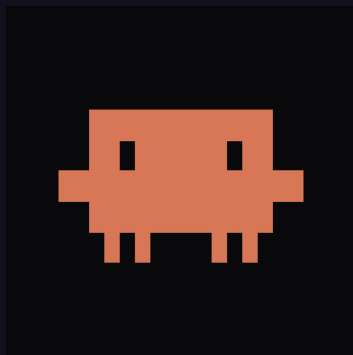


The World is Divided  
Into 2.



People who like to see their code.

*You want the diff. You want the cursor blinking. You want the file tree.*



# People who don't.

*You delegate. You trust the agent. You read the PR, not the file.*

If you're the first type,  
this talk is for you.



# Mor Hananovitz

- AI Engineering Group Lead



- GenAI Lecturer



- WiDS IL Community Leader



Snowboarder, CrossFiter, Boulder Climber

# Agenda

A decorative graphic on the left side of the slide, consisting of several horizontal bars of different colors (green, pink, purple, red, teal, yellow, white, orange, light green, dark grey) arranged in a staggered, overlapping fashion.

01 The Harness

02 Workflows

03 Cost Savings

04 Cursor Automations



01 { ..

# The Harness

Rules, Skills, Commands, Subagents





**HARNESS**





# Four levers, one system

| Tool  | What it is                              | When to use                  |
|-------|-----------------------------------------|------------------------------|
| Rules | Always-on instructions in every context | Team conventions, never-do's |

*rules = passive*    •    *skills = active*  
*commands = shortcuts*    •    *subagents = workers*



# Four levers, one system

| Tool     | What it is                              | When to use                  |
|----------|-----------------------------------------|------------------------------|
| Rules    | Always-on instructions in every context | Team conventions, never-do's |
| Commands | Slash-triggered shortcuts               | Quick, single-action tasks   |

*rules = passive      •      skills = active*  
*commands = shortcuts      •      subagents = workers*



# Four levers, one system

| Tool     | What it is                              | When to use                         |
|----------|-----------------------------------------|-------------------------------------|
| Rules    | Always-on instructions in every context | Team conventions, never-do's        |
| Commands | Slash-triggered shortcuts               | Quick, single-action tasks          |
| Skills   | On-demand markdown playbooks            | Repeatable tasks you invoke by name |

*rules = passive      •      skills = active*  
*commands = shortcuts      •      subagents = workers*



# Four levers, one system

| Tool      | What it is                              | When to use                          |
|-----------|-----------------------------------------|--------------------------------------|
| Rules     | Always-on instructions in every context | Team conventions, never-do's         |
| Commands  | Slash-triggered shortcuts               | Quick, single-action tasks           |
| Skills    | On-demand markdown playbooks            | Repeatable tasks you invoke by name  |
| Subagents | Isolated agents spawned mid-task        | Parallel work, validation, risky ops |

*rules = passive      •      skills = active*  
*commands = shortcuts      •      subagents = workers*

M

mor.h@lsports.eu  
Team Plan

Search settings ⌘F

⚙ General

⚙ VS Code Settings ↗

▽ Agents

→| Tab

📦 Models

☁ Cloud Agents

🔌 Plugins

📄 Rules, Skills, Subagents

🔧 Tools &amp; MCPs

⚡ Hooks

📁 Indexing &amp; Docs

🌐 Network

🧪 Beta

📦 Marketplace

📄 Docs ↗

## Rules, Skills, Subagents

Provide domain-specific knowledge and workflows for the agent

All User stm-recommendation-...

Include third-party Plugins, Skills, and other configs  
Automatically import agent configs from other tools



### Rules ⓘ

[+ New](#)

Use Rules to guide agent behavior, like enforcing best practices or coding standards. Rules can be applied always, by file path, or manually.

### Skills ⓘ

[+ New](#)

Skills are specialized capabilities that help the agent accomplish specific tasks. Skills will be invoked by the agent when relevant or can be triggered manually with / in chat.

pr-code-review

brainstorming



# Rules: the constitution

Live in `.cursor/rules/` – injected silently into every context.

```
# code-review.mdc
```

- Always check for missing error handling
- Flag `console.log` in prod code
- Comment on test gaps, don't just approve

## The trap

Rules are LLM suggestions, not enforcement. The agent can “forget.” That's why the next layers exist.



Live Demo





# Commands: the shortcut layer

Triggered with `/` – lower ceremony than skills, no file. Best for single-step, high-frequency actions.

`/cr`

Review the current PR diff for bugs,  
missing error handling, test gaps.  
Be terse.

**command** = fast &  
disposable

**skill** = structured &  
shareable



<!DOCTYPE html>  
<html style="height:100%">  
<head>

body {  
background: #585858 url(...);  
background-color: #f5f5f5;  
}

<meta name="viewport" content="width=device-width, initial-scale=1">  
<title> Breaking News  
</head>

h1 {  
font-family: Arial, sans-serif;  
font-size: 15px;  
line-height: 1.42857143;

<body style="color: #444;">  
<div style="height: 100px; margin-top: 10px;">  
<div style="text-align: center; margin-top: 10px;">

Live Demo



# Skills: **your** playbook

Markdown the agent fetches on demand – keeps context lean. Invoke by name.

```
# pr-to-cr.md
```

```
## When invoked
```

Given a PR URL, review for:

1. Error handling
2. Security / injection
3. Test coverage

```
## Output
```

Summary + Blockers + Praise



# Subagents: spawn a worker

The parent creates a child with a focused task and limited permissions.

## Validation

Check work against the ticket

## Parallel work

Independent tasks at once

## Risky writes

Isolate destructive ops

```
$ gh pr create → subagent spins up  
reads the diff · applies the CR skill · posts a gh pr comment
```



# Subagents vs. Multitask

|               | Subagent                               | Multitask                               |
|---------------|----------------------------------------|-----------------------------------------|
| What is it    | A child agent spawned by another agent | A separate top-level agent conversation |
| Relationship  | Parent → child hierarchy               | No hierarchy                            |
| Communication | Parent receives child's result         | No cross-task communication             |
| Initiated by  | The agent (via Task tool)              | The user (via Cursor UI)                |
| State sharing | Parent passes context explicitly       | None – fully isolated                   |



02 { ..

## Advanced workflows



} ..



# Hooks: determinism

If X happens, Y always happens. Hooks live outside the LLM – they can't “forget.”

```
{ "hooks": {  
  "afterShellExecution": [{  
    "command":  
      ".cursor/hooks/auto-cr.sh",  
    "matcher": "gh\\s+pr\\s+create"  
  }]  
}
```

## **Always exit 0**

don't crash the agent

## **Untrusted output**

runs with full permissions

## **Mind performance**

heavy hooks block

*The hook returns additional\_context – it injects the next instruction, no human needed.*



Hooks: the use-case menu

*The test: “Am I OK if the LLM sometimes skips this?” If no – it's a hook.*





# Hooks: the use-case menu

## Quality gates

lint, format,  
type-check, tests

## Security & safety

secret scan, block rm  
-rf / force-push

## Auto-review

PR review,  
plan-vs-code check

## Context injection

sprint standards,  
failure output

## Notifications

ping Slack when a run  
finishes

## Cost guards

warn before  
huge-context reads

## Observability

audit every shell  
command

## Cleanup

kill temp files,  
stray console.logs

**Three rules:** fail open · treat output as untrusted · mind performance

*“Rules and skills teach the agent. Hooks are how you stop trusting it to remember.”*



# Plan smart, execute stupid

Plan Mode is **read-only** – browse and analyze, but no writes. Review the plan before execution.

## **X Bad DoD**

“Add logging when file reads fail”

## **✓ Good DoD**

Wrap `readFileContents` in `try/catch`  
`logger.error: [File Read Error] + path`  
re-throw after logging · write unit tests

**Plan expensive** → iterate + cross-check 2nd model → small steps →  
**execute cheap** → **subagent validates**

**“Plan expensive. Execute cheap. Validate automatically.”**



# The Agents Window

Stop alt-tabbing between three Cursor windows. One panel, many agents.

Every session in  
one panel – chat,  
diffs, redirect.  
Local + cloud  
together.

✨ What's up next, Mor?

Default trd-agents-agentic-layer main × trd-agents-litellm main × +



Describe a task or ask a question



New Agent

Automations

Customize

Repositories

vibekode\_demo

Hooks for LLM use-case ...

Subagent vs. multitask c...

Comparison of methods ...

Code review for cursor p...

Code review for demo us...

Code review for cursor p...

Add code review skill

Code and documentatio...

Single character input

Python script for code re...

See more

.github

Automation channel i...

Slack channel context

queen-learning

Refer friends, earn up t...

MH Mor Han

Lsports

Update

Subagent vs. multitask comparison

...

explaibn subagent vs. multitask

Multitask is a **Cursor session setting** that lets the IDE run **multiple agent conversations in parallel** — each is its own top-level agent, not a child of another. It's a UI/session concept, not a code concept:

- Each "task" is a separate chat thread running concurrently
- They don't share state or communicate with each other
- You switch between them in the Cursor UI
- When Multitask Mode is active, any subagents you launch **must** be backgrounded ( `run_in_background: true` ) because the parent agent shouldn't block

### The key distinction

|               | Subagent                               | Multitask                               |
|---------------|----------------------------------------|-----------------------------------------|
| What it is    | A child agent spawned by another agent | A separate top-level agent conversation |
| Relationship  | Parent → child hierarchy               | Peer conversations, no hierarchy        |
| Communication | Parent receives child's result         | No cross-task communication             |
| Initiated by  | The agent (via <code>Task</code> tool) | The user (via Cursor UI)                |
| State sharing | Parent passes context explicitly       | None — fully isolated                   |

In short: subagents are for an agent to **delegate work programmatically**, while multitask is for the user to run **multiple independent agent sessions side by side** in the IDE.

+ Send follow-up

Sonnet 4.6 Medium

Local

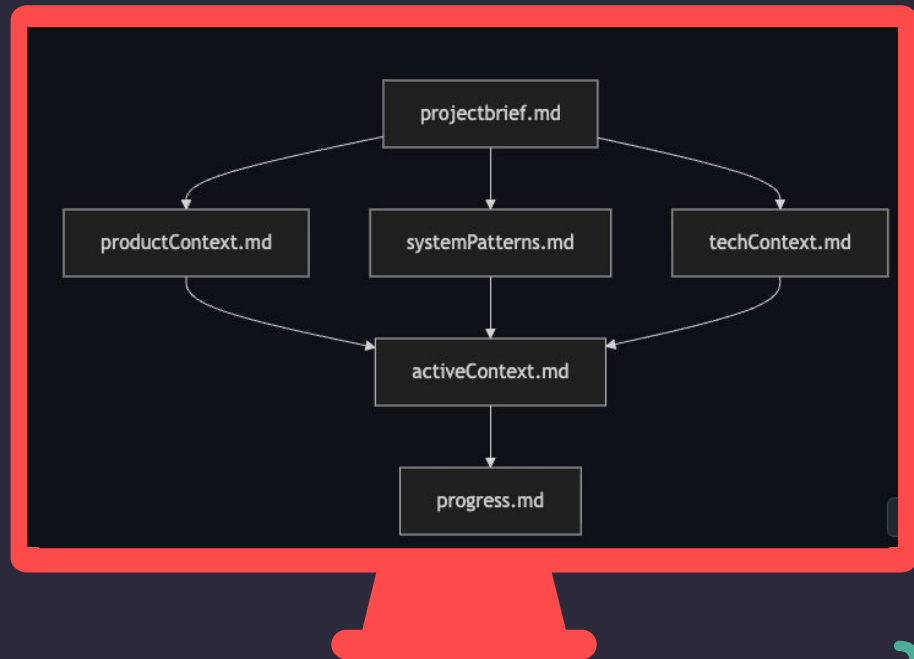
16%

# { Memory bank

Git repo

You tube tutorial

“initialize memory bank”





03 { ..

Money Money Money



} ..

# Model pricing

All prices are per million tokens, sourced from each provider's API pricing:

| Name ^                                                                                                                                                                                | Input  | Cache Write | Cache Read | Output |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|-------------|------------|--------|
|  Claude 4.6 Sonnet  | \$3    | \$3.75      | \$0.3      | \$15   |
|  Claude Fable 5     | \$10   | \$12.5      | \$1        | \$50   |
|  Claude Opus 4.8    | \$5    | \$6.25      | \$0.5      | \$25   |
|  Composer 2.5                                                                                        | \$0.5  | -           | \$0.2      | \$2.5  |
|  Gemini 3.1 Pro                                                                                      | \$2    | -           | \$0.2      | \$12   |
|  Gemini 3.5 Flash                                                                                    | \$1.5  | -           | \$0.15     | \$9    |
|  GPT-5.3 Codex      | \$1.75 | -           | \$0.175    | \$14   |
|  GPT-5.5            | \$5    | -           | \$0.5      | \$30   |
|  Grok Build 0.1     | \$1    | -           | \$0.2      | \$2    |

Show more models

## Auto pricing

| Token type          | Price per 1M tokens |
|---------------------|---------------------|
| Input + Cache Write | \$1.25              |
| Output              | \$6.00              |
| Cache Read          | \$0.25              |





# Choosing the right LLM

It's not a one-model world. A practical decision tree:

Planning, architecture, ambiguity?

→ **Sonnet / Opus**

Execution of a clear, scoped task?

→ **Haiku / local**

Validation against a spec?

→ **Cheapest wins**

Reliable multi-step tool use?

→ **Sonnet is safest**

Latency critical (interactive)?

→ **Haiku**

Accuracy critical (prod code)?

→ **Opus + human review**



# Cost optimization

You're paying for reasoning you don't always need. Three levers:

## Context hygiene

Every token costs. Skills fetch on demand; bloated rules are dead weight.

## Plan cheap, execute cheap

Opus plans, Haiku executes. A well-broken plan means small models won't hallucinate.

## Hooks as cost guards

Lint in a local hook is free. Lint re-run inside the LLM costs tokens every call.



04 { ..

# Cursor Automations



} ..



05 { ..

Runtime



} ..



# Cloud vs local agents

Pick the right runtime, not just the right model.

|                 | Cloud (Cursor, Claude.ai) | Local (Claude Code)    |
|-----------------|---------------------------|------------------------|
| <b>Context</b>  | Managed, limited          | Explicit, full control |
| <b>Security</b> | Code leaves your machine  | Air-gapped option      |
| <b>Cost</b>     | Per-token                 | Infra cost             |
| <b>Speed</b>    | Fast                      | Model-dependent        |
| <b>Hooks</b>    | IDE-level                 | Shell-level, flexible  |

*Callback: the Agents Window shows both runtimes in one view.*



<!DOCTYPE html>  
<html style="height:100%">  
<head>

body {  
background: #585858 url(...);  
background-color: #e6e6e6;  
}

<meta name="viewport" ...>  
<title> Breaking ...  
</head>

h1 {  
font-family: Arial, sans-serif;  
font-size: 15px;  
line-height: 1.42857143;

<body style="color: #444;">  
<div style="height: 100px;">  
<div style="text-align: center;">

# Agent Screen Demo

<h2 style="margin: 0; background-color: #f0f0f0; padding: 5px 10px;">



# The full picture

Everything wired together – one real feature delivery:

1 **Plan (Opus)** – DoD defined

2 **Tasks created** – Jira / Linear

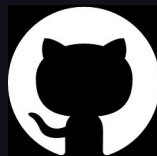
3 **Execute (Haiku)** – small PRs

4 **push → hook** – subagent validates vs ticket

5 **pr create → hook** – pr-to-cr skill posts comment

6 **Human review** – clean & structured

*This is engineering with AI – not vibe-coding.*





# Thanks!

< Do you have any questions? >



CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**